

Group Assignment 2

ECSE 551 – Machine Learning for Engineers – Fall 2025

November 4, 2025

1 General instructions

- **The assignment is due on Nov. 30 at 11:59pm (EST, Montreal Time).**
- Late submissions will be penalized 5 percent per day.
- This assignment is to be completed in groups of three. All members of a group will receive the same grade for the submitted portion of the work except when a group member is not responding or contributing to the assignment. If this is the case and there are major conflicts, please reach out to the contact TA or instructor for help (well before the deadline) and flag this in the submitted report.
- There will be an in-person demo. For this, students will receive an individual mark that depends on how well they answer the questions posed to them.
- Each team member should familiarize themselves thoroughly with all aspects of the submitted solution. The demo questions will NOT be tailored to the specific contributions of each team member.
- Submit your assignment on MyCourses as a group. Any group member can submit, but there is only one submission per group. Submit your assignment as a single Jupyter Notebook (.ipynb) named with the last names of all group members in alphabetical order (e.g., Roy_Smith_Tremblay_Assignment2.ipynb).
- Python should be used for this and all assignments. You are free to use libraries with general utilities, such as matplotlib, numpy and scipy for Python, unless stated otherwise in the description of the task. In this assignment, you are free to use scikitlearn or other package implementations, but you should consider carefully your architectural specifications.

2 Overview

In this assignment, you will participate in an in-class Kaggle competition on image classification. The goal is to devise a machine learning algorithm to successfully classify images in an unseen test set. This lab is about careful design, innovation and practical experimentation, rather than focusing on low-level algorithm implementation.

You may therefore employ library functions and models, but you **may not** copy a full model from another source.

In Task 1, you are required to design a convolutional neural network and a vision transformer and analyze their performance in the task. Task 2 is a free-for-all. Do whatever you like to get the best score.

3 Using Google Colab and Kaggle Competitions

Students are expected to use Google Colab for this assignment, which provides free access to GPUs. To use Colab:

- a. Go to <https://colab.research.google.com/> and create or upload your Jupyter Notebook.
- b. Change the runtime to GPU (Runtime → Change runtime type → GPU).
- c. Execute cells sequentially to run code efficiently with GPU acceleration.
- d. Note that GPU usage on Google Colab is limited. You can also access free GPUs on Kaggle. It is recommended to debug and test your code on CPU or with a few training epochs first, then perform full training on GPU.

A **Kaggle competition** is an online platform for machine learning challenges. Students will train models on a provided dataset and submit predictions to receive a score on a leaderboard. To participate:

- a. Create a free Kaggle account.
- b. Join the competition here (<https://www.kaggle.com/t/28465e848fb84b69bd3214b50f482a96>) and download the dataset.
- c. Train models on the training set and submit predictions for the test set.

4 Dataset Description

The dataset for this assignment is organized as follows:

- Training set: 10,000 images with labels corresponding to 10 classes.
- Test set: 2,000 images without publicly visible labels. A fraction (30%) of the test set is used as a public test set for the competition leaderboard, while the remaining 70% is the private test set, used for final evaluation after the submission deadline.

For conducting the detailed performance analysis in Task 1, you should create your own “test” set by dividing the provided training data into a Training-Test split. It is advised that you also create a validation set (or perform cross-validation) to select hyperparameters.

5 Task 1

- a. Design and implement (a) a CNN; and (b) a Vision Transformer. Apply them in the Kaggle competition.
- b. In the report, justify all of your design decisions and explain the operation of your models.
- c. Explain your training process. Specify data splits. Explain how you selected any hyperparameters.
- d. Report your results, both in training, validation, and test.
- e. Conduct a comparison of performance that goes beyond merely checking accuracy. Are there specific categories where performance is poorer? Consider a contingency table that explains how often both classifiers are correct, how often both are wrong, and in how many cases one is right and the other is wrong.

Some Tips and Tricks

To improve the performance of your models, you are encouraged to apply data preprocessing (e.g., normalization, standardization) and data augmentation techniques (e.g., random rotations, flips, crops, color jittering). These help improve generalization and reduce overfitting.

For Task 1, you are expected to implement both models from scratch (i.e., define their architecture manually), although you may use standard PyTorch building blocks such as `nn.Conv2d`, `nn.ReLU`, `nn.MaxPool2d`, or `nn.Linear` for the CNN, or `nn.TransformerEncoderLayer`, `nn.LayerNorm`, `nn.Parameter`, `nn.Dropout`, or `nn.Linear` for the ViT. This list is not exhaustive, and you may use other PyTorch modules as needed to implement your models. However, you may not directly import or instantiate pre-built CNN or Vision Transformer models.

You must select an appropriate loss function (e.g., `nn.CrossEntropyLoss()`) and optimizer (e.g., `optim.Adam(model.parameters(), lr=...)`) and justify your choices. Clearly describe your training strategy, including batch size, learning rate schedule (if used), and stopping criterion (e.g., early stopping based on validation loss). Report any regularization techniques (dropout, weight decay, etc.) or learning rate adjustments that contributed to performance improvements.

Finally, ensure your training code saves the best-performing model based on validation results, and that your final test predictions are generated using this model. For example, you can save your model's weights using PyTorch:

```
torch.save(model.state_dict(), "vit_weights.pth")
```

You can later load the saved weights with:

```
model = ViT() # instantiate a new model with the same architecture
model.load_state_dict(torch.load("vit_weights.pth"))
```

Make sure your ViT and CNN classes defines both a `__init__` method and a `forward(self, x)` method. You should create your class instance as an `nn.Module` (e.g. `class ViT(nn.Module):`), which allows you to access built-in PyTorch functionality.

Please submit both your code and the model weights corresponding to the final results. We will verify your results, and any mismatch between the submitted weights and your architecture may lead to incorrect evaluation. Since you will be coding on Colab, it is good practice to save checkpoints regularly using `torch.save(state_dict)`.

6 Task 2

In Task 2, you may use any learning architecture of your choice, including ensembles of models, to obtain the best score on the test set for the Kaggle competition. Partial grades will be assigned based on your ranking in the competition and your achieved accuracy compared to the best and worst scores.

In the report, you should provide a brief description of the architecture you employed in Task 2 and any details you consider important regarding its operation and training. It is **not** expected that you provide a detailed design or performance analysis.

7 Deliverables

Submit two separate files to myCourses. Please use consistent naming conventions (surnames in alphabetical order).

- A single Jupyter Notebook (.ipynb) named with the last names of all group members in alphabetical order (e.g., Roy_Smith_Tremblay_Assignment1.ipynb). Your notebook should reproduce all of the results included in your report. Please ensure that all original training and output results are saved in your notebook. These should be the same results provided in the report.
- The weights of your CNN, ViT and Task 2 model.
- A (max. 8 page) report submitted as a pdf. This page length INCLUDES all figures, references and appendix. (Any material after 8 pages will be ignored during grading).

7.1 Report

Please prepare your document using one of the templates provided here (we highly recommend preparing the report using latex):

<https://www.ieee.org/conferences/publishing/templates>

The report should include the following sections:

- **Abstract:** a very brief (100-150 words) summary of the content of the report.
- **Introduction:** summarize the assignment tasks, the datasets, and the most important findings.

- **Methodology/Design:** Describe the architectures you developed for the CNN and ViT. Provide details of the design process and specify the training methodology. Explain how you selected hyperparameters. For Task 2, provide a brief description of the architecture you employed.
- **Datasets:** Very briefly describe the datasets and any preprocessing you performed. Include any statistical analysis of the data that you deem most important.
- **Results:** Together with methodology, this should be the longest section of the report. Describe the results of the experiments you conducted in Task 1 to understand and compare the performance of the CNN and the ViT. Discuss how these results influenced any design decisions. More in-depth analysis of the performance is very welcome. For Task 2, you only need to provide very brief text summarizing results. You may include tables and/or figures.
- **Discussion and Conclusion:** Summarize the key findings of the work and suggest any future directions to improve performance.
- **Statement of contributions:** State the breakdown of work among the team.

7.2 Evaluation

A breakdown of the grading of the assignment is as follows: Report and code: 40 percent; demo: 40 percent; competition: 20 percent.

Report:

- a. **Completeness (10/40)** – did you submit all required components? Did you execute all required experiments? Did you follow the guidelines for the report?
- b. **Correctness (20/40)** – are the models implemented correctly? Do you observe reasonable trends in your analysis? Is your interpretation of the results correct and do you provide adequate justification?
- c. **Writing quality (10/40)** – is the report free of grammatical errors and typos? Is the English usage satisfactory? Is the writing clear and easy to understand? If you include them, are figures and tables correctly presented? Do you appropriately cite referenced work? Are your citations correctly formatted?

Demo:

Each member of the team will be asked questions for approximately 5 minutes about the entire submission (not just their own contribution). The demo grade for each student will be assigned based on the understanding of the code, the algorithms, and the results.

Competition:

Marks will be assigned based on the rank of your group in the class and the performance of your proposed technique compared to the best and worst submissions.